

Contenedores y Capas

De qué trató la segunda clase, con ejercicios para que los conceptos se queden. Esta transcripción sí se entiende bien, así que aquí no hay nada que descifrar — solo que ordenar. Las tareas están hasta abajo, en una lista que puedes ir marcando.

El recorrido de la clase

- 00:00 Los manda a la página de ESCOM a ver el **minisitio de nuevo ingreso**, y les deja clonarlo como tarea.
- 03:00 **Contenedores**: por qué no basta con virtualizar, qué es Docker, imágenes, Compose.
- 13:52 Comandos en vivo: `docker run`, el archivo `docker-compose.yml`, y el mapeo de puertos.
- 26:15 Su pregunta trampa: "**¿cuántos contenedores llevamos instalados?**"
- 36:22 **Arquitectura de N capas** y cómo se ve en el ecosistema Java tradicional.
- 53:19 **Arquitectura de 3 capas**, capas lógicas vs. físicas, y por qué "el tiempo real no existe".
- 64:30 Tareas, equipos y los **entregables del primer parcial**.

PARTE 1

Contenedores

Corresponde al punto 1.2.1 del temario. Es el tema con el que van a trabajar en las próximas clases, así que conviene que quede firme.

El problema que resuelven

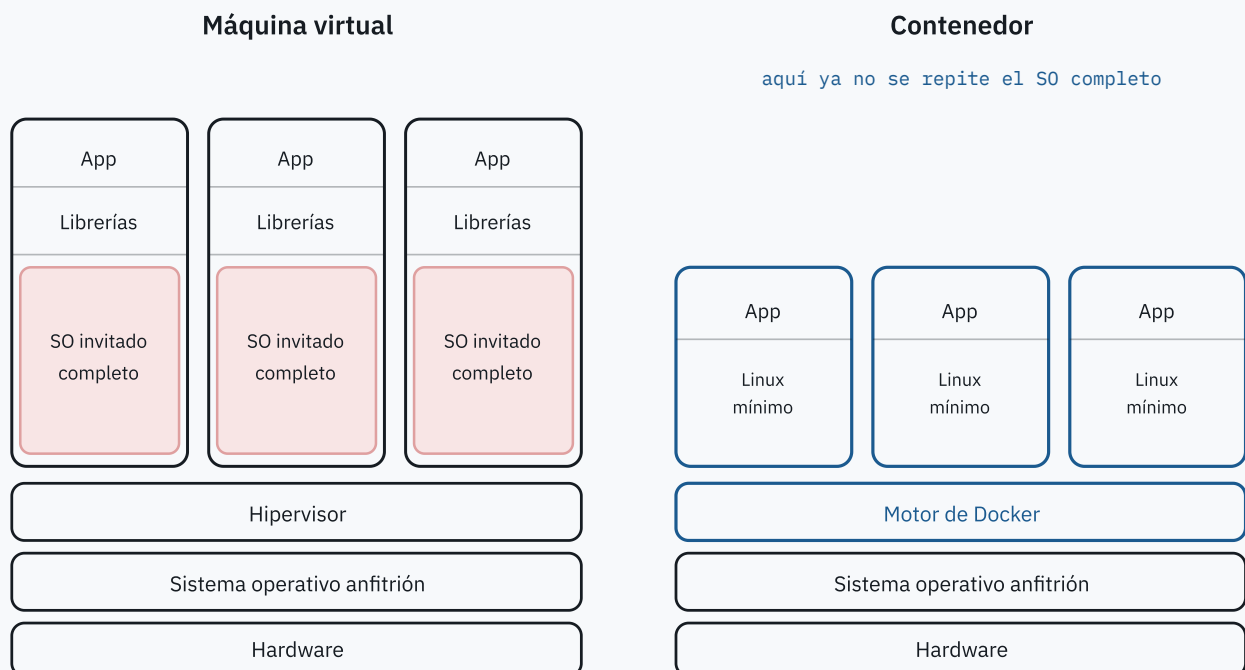
Arrancó con la costumbre normal: tengo Windows, quiero un Linux, y para no arruinar la máquina lo **virtualizo**. Funciona, pero una máquina virtual sigue siendo pesada — es una partición, una unidad completa, y consume los mismos recursos de tu máquina.

Y planteó el caso concreto: **instala Microsoft SQL Server en un Linux**. No se puede así nomás. En sus palabras, Microsoft "siempre ha sido celoso": Microsoft con Microsoft, sobre Windows. Tendrías que virtualizar un Windows entero... para una práctica que vas a usar una semana y luego olvidar.

"¿Pa' qué lo instalo? ¿Pa' qué virtualizo? ¿Pa' qué particiono?" — ahí es donde entra el contenedor.

Un contenedor es un **espacio aislado** con su propio sistema operativo base mínimo y su propio entorno de ejecución, que te deja montar software que de otro modo no sería compatible. Le llamó **virtualización ligera**. Y el otro problema que resuelve es el clásico:

"Necesito evitarme el típico 'es que en mi máquina funciona'. Sí, por supuesto que en tu máquina funciona, porque le instalaste alguna dependencia y ya ni te acuerdas por qué la necesitabas."



La diferencia que importa es el bloque rojo: cada máquina virtual arrastra un sistema operativo invitado completo. El contenedor no — comparte el anfitrión y solo lleva lo mínimo. Por eso arranca en segundos y por eso puedes tener tres bases de datos distintas conviviendo sin que se estorben.

Su analogía fue la de los **contenedores de barco y ferrocarril**: un espacio que le corresponde solo a ese proceso, con todo lo necesario adentro para funcionar. Y formalmente lo definió así: **un contenedor es una instancia ejecutable de una imagen**.

- **Imagen** — el paquete congelado: tu proyecto más todo lo que necesita. La construyes, la subes al hub, alguien la descarga.
- **Contenedor** — esa imagen ya corriendo.
- **Docker vs. Podman** — mencionó Podman como alternativa. Todo lo que haces en Docker lo puedes hacer en Podman; Podman es más de comunidad y enfocado a sistemas tipo Red Hat / RHEL. Van a usar Docker porque es más intuitivo.

- **No está anclado a un sistema operativo** — la misma imagen corre en Windows, Linux y Mac.
- **La configuración va en variables de entorno** — cadena de conexión, usuario, contraseña. No se hornean dentro de la imagen.

El comando que más van a escribir

Este fue su ejemplo para levantar un servidor web. Píchale a cada pedazo para ver qué hace:

PRACTICA · 1 DE 6

Desarma el comando

Cada parte tiene una razón de ser. Toca los pedazos azules.

```
docker run -d -it --name apache-web-server -p 8080:80 httpd
```

Toca cualquier parte del comando para ver qué significa.

Los puertos: la parte que más se confunde

Dedicó un buen rato a esto porque es donde todo el mundo se equivoca. En

`-p 8080:80` hay dos puertos distintos y el orden importa.

PRACTICA · 2 DE 6

¿Por cuál puerto entras tú?

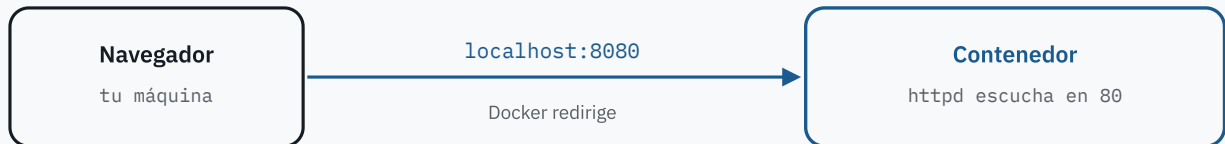
Levantaste el Apache con `-p 8080:80`. Abres el navegador. ¿Qué escribes?

localhost:8080

localhost:80

Cualquiera de los dos, es lo mismo

-p 8080:80 — izquierda es tuya, derecha es del contenedor



Para levantar tres bases de datos a la vez, cambias solo el número de la izquierda:

`3306:3306` • `3307:3306` • `3308:3306`

El número de la derecha es fijo — es el puerto por el que ese programa escucha siempre. El de la izquierda lo eliges tú, y es lo que te permite tener varias copias del mismo servicio sin que choquen.

docker-compose.yml

Cuando necesitas varios servicios a la vez —base de datos, backend, frontend— ya no escribes comandos sueltos: los declaras en un archivo y levantas todo con `docker-compose up`.

```
services:
  mysql:
    image: mysql:8.0
    container_name: mi-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: secreto
      MYSQL_USER: alumno
      MYSQL_PASSWORD: otracosa
    ports:
      - "3306:3306"
```

Detalles que mencionó: la línea `version:` ya es opcional y si la pones te sale un *warning*. Soporta **volúmenes**, recomendados cuando va a haber persistencia de datos. Y `depends_on` sirve para garantizar el orden — que la base de datos esté viva antes de que arranque el backend.

El detalle que te puede costar el examen

Dijo textualmente que si se lo entregan mal les pone "su tachezote".

¿Cuál de los dos sirve?

OPCIÓN A

```
services:
  mysql:
    image: mysql:8.0
    ports:
      - "3306:3306"
```

OPCIÓN B

```
services: mysql: image: mysql:8.0 ports: "
```



Los comandos para ver qué está pasando

- `docker images` — qué imágenes ya tienes descargadas. Si pides una versión que ya bajaste, la usa local; si no, se conecta a internet. Por eso el primer uso sí necesita red.
- `docker ps` — qué contenedores están corriendo: id, imagen, estado, puertos, nombre.
- `docker exec -it apache-web-server bash` — te conecta a la consola dentro del contenedor, para configurarlo desde adentro.

Sobre los `0.0.0.0` que aparecen en la columna de puertos de `docker ps`: significan que está escuchando por **todas** las interfaces de red, para que cualquiera pueda llegar. Y avisó que eso lo van a necesitar entender pronto — probablemente cuando conecten sus máquinas entre sí en el laboratorio.

Aparte les dejó una corrección de vocabulario: "Apache" es el nombre de la fundación, no del servidor. El servidor web es el proyecto httpd. Tomcat es otro proyecto distinto de la misma fundación. "De cariño le decimos Apache 2 los que jugamos con Unix."

Su pregunta trampa

A media clase preguntó esto "nomás para ver si están despiertos".

Piénsalo antes de contestar.

"¿Cuántos contenedores llevamos instalados hasta aquí?"

Ninguno

Uno

Dos

Tres

PARTE 2

Arquitectura de N capas

Retoma la Unidad I donde se quedó la clase pasada. Corresponde al punto 1.3 del temario.

Empezó por definir la N con dos límites: **N significa más de tres**. No pueden ser tres, porque a esa se le llama justamente "arquitectura de tres capas". Y no pueden ser dos, porque eso sería el clásico cliente-servidor — aunque aclaró que, por definición, **todo lo que hacemos para web es cliente-servidor**.

Dijo que las arquitecturas en capas son de las más usadas por su simplicidad, y fue honesto sobre el motivo real: **es la que usas por defecto cuando no tienes idea de cuál usar**.

Sobre MVC aclaró algo que suele confundir: es un *patrón de diseño*, no una arquitectura — de composición o arquitectónico según el autor. Y que MVC y la de tres capas son muy parecidas: "solo es muy semántica la definición".

Pero para el trabajo de ellos, dijo que necesitan **al menos cuatro elementos**:



Dos direcciones opuestas: para usarla vas de arriba abajo; para levantarla, de abajo arriba.

Esta doble dirección es el punto que más repitió. La petición viaja hacia abajo, pero el orden en que enciendes las cosas es el inverso: sin base de datos viva, la persistencia no responde, y así en cascada hacia arriba.

Arma el orden de arranque

Vas a desplegar la aplicación desde cero. ¿Qué levantas primero? Toca las capas en el orden correcto.

1.

2.

3.

4.

Base de datos

Lógica de negocio

Persistencia

Presentación

Reiniciar

Cómo se ve esto en el mundo Java

Aquí dibujó el ecosistema Java tradicional completo, y es donde el concepto de "capas" deja de ser abstracto: cada capa es literalmente **un servidor distinto**, y cada uno entiende cosas distintas.

¿Quién entiende qué?

Cada pieza del stack Java entiende un tipo de cosa y le pasa el resto al siguiente. Asigna cada tecnología a quien la atiende.

HTML, CSS y JavaScript

Elige...



Una JSP (que técnicamente es un Servlet)

Elige...



EJBs y Jakarta MS (mensajería)

Elige...



Las tablas y los registros

Elige...



Revisar

Reiniciar

Y le puso números al asunto, que es la parte que nadie menciona en clase normalmente: un hospedaje web sencillo son **unos 300 pesos al año**. Algo que entienda Java ya requiere un VPS — **unos 500 pesos al mes**. Y un servidor con soporte para componentes empresariales, en sus palabras, "carísimo".

Cerró esa parte con el motivo de que exista Spring Boot: **evitar toda esa complejidad**. También mencionó que Tomcat sacó una línea llamada TomEE (EE de Enterprise) para aplicaciones Java empresariales.

PARTE 3

Arquitectura de tres capas

La versión estricta: ni una capa más, ni una menos.

Las mismas tres de siempre — **presentación, lógica de negocio y acceso a datos** — pero aquí el número es exacto. La de presentación puede ser una app web, de escritorio o móvil; eso no cambia el conteo.

Sobre la **capa de acceso a datos** señaló una ambigüedad honesta: dependiendo del autor, la base de datos misma está en esa capa, o solo viven ahí las clases que representan a las tablas. No hay una respuesta única, y conviene saberlo por si el examen la toca.

En Java hay dos formas de acceder a datos: **JDBC** —la de la lectura que les dejó— y un **ORM**, que puede ser JPA o Hibernate. Y aclaró cómo se reparten el trabajo hoy:

*"Todos los decoradores bonitos son de **JPA**, todo el manejo de las transacciones y operaciones le toca al **Hibernate**." Desde la versión 6 están homologados.*

Capas lógicas vs. físicas: si todo está en el mismo equipo son lógicas. Si están en máquinas o contenedores distintos, son físicas. Su ejemplo fue de lo más concreto — la base de datos en la laptop de un compañero, el backend en la de otro, el frontend en la de otra.

Un aparte que vale la pena: "el tiempo real no existe"

Al hablar de rendimiento se desvió a algo que le da curiosidad de los trabajos terminales: la gente que escribe "*aplicación en tiempo real*" en el título.

*"Todos van a coincidir en que **el tiempo real no existe**. Y hay un motivo: en todo mecanismo de comunicación hay **latencia**."*

Su argumento concreto: hasta Socket.IO —la herramienta que todos citan para esto— documenta un porcentaje de pérdida. No te da el 100%. Concedió que en un libro de sistemas operativos sí existe una clasificación formal de sistemas de tiempo real, pero que **hay que justificarlo**, no solo ponerlo en el título.

Y el punto que le quita presión a su proyecto

Al hablar de que las capas suelen estar amarradas a un solo stack, llegó a una conclusión que conviene tener presente cuando armen el proyecto:

*"Como nosotros no vamos a hacer cosas para web tradicional, eso no aplica. **Nosotros vamos a hacer APIs y servicios**. Y al ser un servicio y al ser un API, en realidad da lo mismo con cuál hierba lo quieran conectar."*

Es decir: el backend en Java/Spring y el frontend en Angular o React *no* son el mismo stack, y eso está bien — precisamente porque los va a unir una API REST y no un framework de web tradicional. Es la misma idea con la que cerró la clase pasada.

Lo último de esta parte, **tolerancia a fallos**: si una capa se cae, todas las que estén encima se caen con ella. "Se cayó la base de datos, pues con la pena: no hay lógica, tú no vas a ver nada."

¿Se te quedó?

Seis preguntas sobre lo que se vio hoy. Sin trampa — bueno, una sí.

1 ¿Qué significa la N en "arquitectura de N capas"?

Más de tres

Cualquier número, incluido dos

Exactamente tres

Número indefinido de servidores

2 Si se cae la capa de acceso a datos, ¿qué pasa con las demás?

Fallan todas las que están por encima

Solo falla esa capa

Falla únicamente la base de datos

El sistema sigue igual

3 "Apache" es, hablando con propiedad...

El nombre de la fundación; el servidor web es httpd

El nombre del servidor web

Lo mismo que Tomcat

Un sistema operativo

4 ¿Existen las aplicaciones en tiempo real?

No como se suele decir: siempre hay latencia

Sí, con Socket.IO se logra el 100%

Sí, en cualquier app web moderna

Solo en aplicaciones móviles

5 Desde la versión 6, ¿cómo se reparten JPA e Hibernate?

JPA mapea entidades, Hibernate maneja transacciones

Hibernate mapea, JPA maneja transacciones

Son lo mismo

JPA reemplazó a Hibernate

6 Dos capas en máquinas distintas son capas...

Físicas

Lógicas

Virtuales

Independientes

Calificar

Otra vez

PENDIENTES

Todas las tareas de la clase

Doce cosas salieron en hora y cuarto. Márcalas conforme las vayas cerrando — se guardan en este navegador.

0 / 12

PARA YA (LO PUEDE PEDIR CUALQUIER DÍA)

☐ **Clonar el minisitio de nuevo ingreso de ESCOM**

Copia fiel en local: mismas figuras, mismo texto, mismo color, mismo audio, mismo video, mismo mapa. Después lo van a servir desde un contenedor en localhost:3500, y en el laboratorio se conectarán a la IP del compañero — por eso hay que cambiarle el título a "Mi minisitio de <tu nombre>".

El día que se le ocurra

☐ **Instalar Docker**

En Windows o Mac, Docker Desktop y listo — carga con todo. En Linux, instalar Docker y verificar que Compose venga incluido, porque en algunas distros va por separado.

Antes de la próxima clase

☐ **Guardar los tres ejemplos de contenedores**

Postgres y Apache por línea de comandos con docker run, y MySQL vía docker-compose. Dijo textualmente "ténganlos a la mano por cualquier cosa".

Referencia

LECTURAS DE LA PLATAFORMA

☐ **Leer "Lectura rápida Java"**

58 páginas. Ya preguntó en clase si la habían leído.

Pendiente desde la clase 1

☐ **Leer "Lectura rápida JDBC"**

Unas 60 páginas. La mencionó otra vez al hablar de acceso a datos: "el JDBC que no han leído".

Pendiente desde la clase 1

INVESTIGACIÓN

☐ **Investigación: arquitecturas limpias y de cebolla**

Puntos 1.3 a 1.5 del temario. Preguntó un alumno si era solo investigación y confirmó que sí: qué es, para qué sirve, con qué se come y cómo se adereza. Van a encontrar una estructura circular de capas.

Para mañana

PROYECTO — PRIMER PARCIAL

☐ **Armar equipos y definir el proyecto**

Equipos de 4, ustedes eligen la problemática. Tienen la semana: el viernes no hay clase, así que son tres días para pensarlo.

Esta semana

☐ **Modelo entidad-relación**

El de rombos, elipses y circulitos. Ojo: lo que genera MySQL Workbench NO es entidad-relación, es la representación relacional de las tablas. Van a entregar ambas cosas.

Avance 1er parcial

☐ **Script SQL**

Asumiendo MySQL, "a menos que se me ocurra lo contrario".

Avance 1er parcial

☐ **Diccionario de datos**

El formato es libre: el que les guste o el que encuentren en la red. Sirve para levantar requerimientos con el cliente — por ejemplo, dejar asentado que idCurso es llave primaria, numérica, autogenerada y obligatoria.

Avance 1er parcial

JEFE DE GRUPO

☐ **Pasarle la lista de firmas**

Lo pidió directamente como una de las tareas.

Pendiente

☐ **Recordarle que suba el pase de lista**

Reconoció que no lo ha puesto y pidió que se lo recuerden: "es su chamba, recuérdense, lo busco y lo pongo".

Pendiente

Sobre las fechas: fue deliberadamente vago con el minisito — "para mañana, para el lunes, para el miércoles o para el jueves de la próxima semana, el día que se me ocurra". La lectura de eso es que lo puede pedir en cualquier momento, así que conviene tenerlo listo ya. Lo de arquitecturas limpias sí lo fijó "para mañana".